



LANGAGE GO

Compte rendu

Ethan BOURGUIGNEAU
M2 IW

Table des matières

Séance 29-05-2026	2
Exercice Noté 1 – Calculateur d’IMC et de catégorie	2
Exercice Noté 2 – Calculatrice avec closures et validation.....	3
Exercice Noté 3 - Système de contacts.....	4
TP Noté 4 – TechShop – Boutique CLI.....	5
Exercice – Slice / Boucle for / Iota (Créatif).....	6
Cours notions	6
Explication des fonctions	6
Fonctions variadiques (variadic functions)	7
Closure en GO	7
Boucle for en GO	8
fallthrough dans switch	8
Slices et copie	9
Structs et POO en Go	11
Struct Tags – métadonnées pour JSON et plus	12
Visibilité en GO	12
Defer – garantir l’exécution d’une fonction	13
Package standard incontournables	13

rcanetti@myges.fr

Séance 29-05-2026

Rappel de ce qu'on avait vu la dernière fois lors du cours du 26-05-2026

Ariane 5 => bug

500 millions de dollars

Explication du array slice :

- Avantage (taille dynamique peut augmenter)

Explication sur l'initialisation d'un ou plusieurs variables

Exercice Noté 1 – Calculateur d'IMC et de catégorie

 **EXERCICE NOTÉ 1 — Calculateur d'IMC et de catégorie** [15 min]

Contexte : L'IMC (Indice de Masse Corporelle) est utilisé en médecine. C'est un premier exercice pratique mêlant types, variables et constantes.

1. Déclarez deux variables float64 : poids (ex. 70.5 kg) et taille (ex. 1.75 m)
2. Créez 4 constantes : IMCMaigre=18.5, IMCNormal=25.0, IMCSurpoids=30.0
3. Calculez l'IMC = poids / (taille * taille)
4. Affichez l'IMC avec 2 décimales (fmt.Printf avec %.2f)
5. Affichez la catégorie : Maigre / Normal / Surpoids / Obésité
6. BONUS : déclarez une constante Nom avec votre prénom et personnalisez l'affichage

Rendu : fichier imc.go sur GitHub

Indices : `fmt.Printf("IMC : %.2f\n", imc) | if/else if/else | const (A = 1.0; B = 2.0)`

```
Run go build exercice1/ x
Votre IMC est : 23.02
Catégorie : Normal
Nom : Ethan
Prénom : Bourguigneau
Process finished with the exit code 0
```

Exercice Noté 2 – Calculatrice avec closures et validation

EXERCICE NOTÉ 2 — Calculatrice avec closures et validation [20 min]

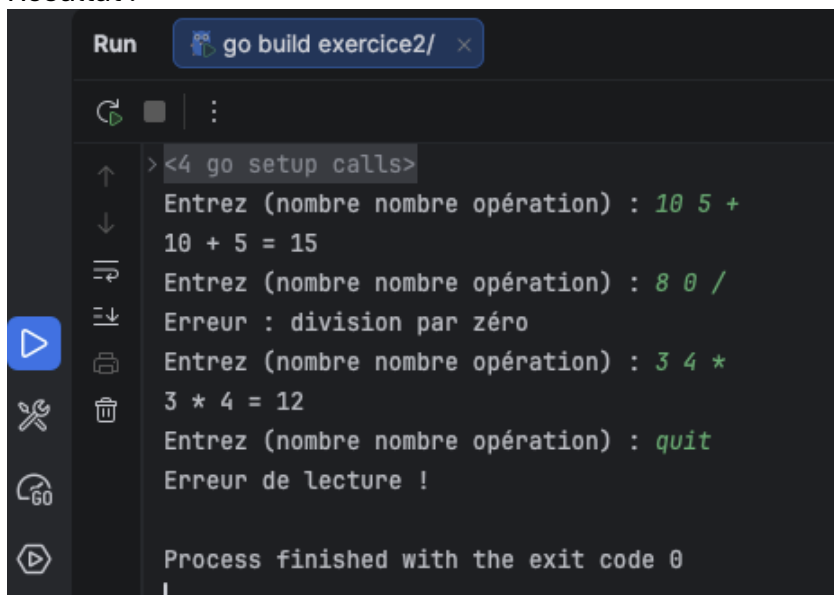
Contexte : Une calculatrice CLI qui démontre les fonctions, retours multiples et closures.

1. Créez une fonction `operer(a, b float64, op string) (float64, error)` qui supporte '+', '-', '*', '/' — retourne une erreur pour / par 0 ou op inconnue
2. Créez une fonction `creerOperation(op string) func(float64, float64) float64` qui retourne une closure pour chaque opération
3. Dans `main()`, faites une boucle 'while true' qui :
 - Lit deux nombres et une opération depuis l'entrée standard (`fmt.Scan`)
 - Affiche le résultat ou l'erreur
 - Sort si l'opération est 'quit'
4. Testez avec : `10 5 +` → 15 | `8 0 /` → erreur | `3 4 *` → 12

Rendu : fichier `calculatrice.go` sur GitHub

Indices : `switch op { case '+': ... } | fmt.Scan(&a, &b, &op) | for { if op == 'quit' { break } }`

Résultat :



```
Run go build exercice2/ x
> <4 go setup calls>
Entrez (nombre nombre opération) : 10 5 +
10 + 5 = 15
Entrez (nombre nombre opération) : 8 0 /
Erreur : division par zéro
Entrez (nombre nombre opération) : 3 4 *
3 * 4 = 12
Entrez (nombre nombre opération) : quit
Erreur de lecture !

Process finished with the exit code 0
```

Exercice Noté 3 - Système de contacts

EXERCICE NOTÉ 3 — Système de contacts : Personne, Employé, Étudiant [20 min]

Contexte : Modéliser un annuaire d'entreprise avec différents types de contacts.

1. Créez une struct `Personne` { `Prenom`, `Nom` string; `Age` int; `Email` string } avec la méthode `NomCompleet()` string et `Presentation()` string
2. Créez `Adresse` { `Rue`, `Ville`, `CodePostal` string } avec `Format()` string
3. Créez `Employe` en embeddant `Personne` et `Adresse` + champs `Poste` string, `Salaire` float64
Ajoutez la méthode `FicheEmploye()` string qui affiche toutes les infos
Ajoutez `AugmenterSalaire(pct float64)` qui modifie le salaire
4. Créez `Etudiant` en embeddant `Personne` + champs `Promo` string, `Moyenne` float64
Ajoutez la méthode `MentionObtenue()` string (TB/B/AB/P selon moyenne)
5. Dans `main()`, créez 2 employés et 2 étudiants, affichez leurs fiches

Rendu : `contacts.go` sur GitHub

Indices : (`e *Employe`) pour modifier le salaire | `switch { case moy >= 16: ... } | fmt.Sprintf pour construire des chaînes`

```
Run go build exercice4/ x
><4 go setup calls>
--- Employee 1 ---
Employé:
Nom: Bob Jane
Âge: 30
Email: bob.jane@gmail.com
Adresse: Rue Tronchet, Lyon (69006)
Poste: développeur web
Salaire: 2500.00€

--- Employee 2 ---
Employé:
Nom: Alice Martin
Âge: 28
Email: alice@mail.com
Adresse: Rue Victor Hugo, Paris (75001)
Poste: DevOps
Salaire: 3200.00€

--- Augmentation Salaire employee 1 ---

--- Employee 1 après augmentation ---
Employé:
Nom: Bob Jane
Âge: 30
Email: bob.jane@gmail.com
Adresse: Rue Tronchet, Lyon (69006)
Poste: développeur web
Salaire: 2750.00€

--- Étudiants ---
Jean Duchemin → B
Tom Lee → AB

Process finished with the exit code 0
```

TP Noté 4 – TechShop – Boutique CLI

EXERCICE NOTÉ TP1 — TechShop — Boutique CLI [45 min autonome + rendu GitHub]

Contexte : Vous êtes développeur chez TechShop, une boutique en ligne de matériel informatique. Vous devez créer un gestionnaire de catalogue en ligne de commande.

=== DONNÉES ===

- Struct Produit : ID int, Nom string, Marque string, Prix float64, Stock int, Catégorie string (actif bool)
- Struct Catalogue avec une slice de Produits

=== FONCTIONNALITÉS (méthodes sur Catalogue) ===

- AjouterProduit(p Produit) error — erreur si ID dupliqué
- TrouverParID(id int) (Produit, error)
- TrouverParCategorie(cat string) []Produit
- AppliquerReduction(categorie string, pct float64) int — retourne nb de produits modifiés
- Vendre(id int, qte int) error — réduit le stock, erreur si insuffisant
- Rapport() string — résumé : nb produits, valeur totale du stock

=== MAIN ===

- Menu CLI interactif : [1] Ajouter [2] Chercher [3] Soldes [4] Vendre [5] Rapport [0] Quitter
- Pré-remplissez 5 produits high-tech réalistes (iPhone, MacBook, etc.)
- Gérez toutes les erreurs proprement

=== RENDU ===

- Fichier : tp1_techshop.go (tout dans un seul fichier)
- Commande : git init + git add + git commit + git push sur votre GitHub
- URL du repo à envoyer dans le chat Teams

=== CRITÈRES D'ÉVALUATION ===

- /5 : Structs + méthodes correctes (value vs pointer receiver)
- /5 : Gestion des erreurs complète
- /5 : Menu CLI fonctionnel
- /5 : Code propre, noms explicites, package main correct

Indices : `fmt.Scan(&choix)` | `for i, p := range c.produits { if p.ID == id { ... } }` | `strings.EqualFold` pour comparaison insensible à la casse

Exemple d'exécution du projet TechShop que j'ai implémenté :

```
Run go build exercice5/ x
<4 go setup calls>

--- TechShop Catalogue ---
[1] Ajouter [2] Chercher [3] Soldes [4] Vendre [5] Rapport [0] Quitter
Votre choix : Votre choix : 5

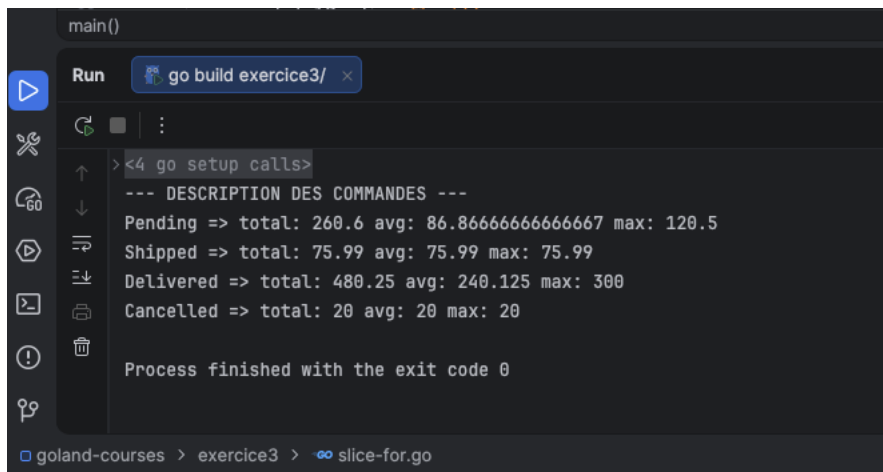
Produits : 5 | Valeur totale du stock : 87199.12€

--- TechShop Catalogue ---
[1] Ajouter [2] Chercher [3] Soldes [4] Vendre [5] Rapport [0] Quitter
Votre choix : Votre choix : 2
[1] Par ID [2] Par catégorie
Votre choix : 2
Catégorie : Smartphone

ID    Nom           Marque    Prix    Stock
-----
1     iPhone 17 Pro Apple     1199.99€ 25
3     Galaxy S25 Ultra Samsung   1099.99€ 15
```

Exercice – Slice / Boucle for / Iota (Créatif)

Gestion de commandes avec des statuts différents.



```
main()
Run go build exercice3/ x
><4 go setup calls>
--- DESCRIPTION DES COMMANDES ---
Pending => total: 260.6 avg: 86.86666666666667 max: 120.5
Shipped => total: 75.99 avg: 75.99 max: 75.99
Delivered => total: 480.25 avg: 240.125 max: 300
Cancelled => total: 20 avg: 20 max: 20

Process finished with the exit code 0
goland-courses > exercice3 > slice-for.go
```

Cours notions

Explication des fonctions

1. Explication générale des fonctions

En Go, une fonction peut :

- Prendre des paramètres
- Retourner une ou plusieurs valeurs
- Retourner aussi une erreur

2. Différence try/catch (Java) vs gestion d'erreurs (Go)

En Java, on utilise try/catch.

En Go, il n'y a pas d'exception classique pour le flux normal : on retourne une error comme valeur.

Différences entre le try catch JAVA et l'approche GO avec l'exemple :

```
func diviser(a, b float64) (float64, error) {
    if b == 0 {
        return 0, fmt.Errorf("division par zéro")
    }
    return a / b, nil
}
```

```
result, err := diviser(10, 0)
if err != nil {
    fmt.Println(err)
    return
}
```

3. Gestion des erreurs avec _

Le _ permet d'ignorer une valeur retournée.

Exemple :

```
result, _ := diviser(10, 2)
```

À utiliser avec prudence, car on perd l'information d'erreur.

Fonctions variadiques (variadic functions)

C'est une fonction variadique qui peut accepter un nombre variable d'arguments. C'est pratique pour avoir des fonctions plus flexibles par exemple quand on ne connaît pas à l'avance le nombre exact de valeurs à traiter.

Syntaxe :

```
func somme(nums ...int) int {  
    total := 0  
    for _, n := range nums {  
        total += n  
    }  
    return total  
}
```

Exemple :

```
somme(1, 2, 3)
```

```
somme(10, 20)
```

Utile quand on ne sait pas le nombre d'entrées à l'avance.

Closure en GO

C'est une fonction appelé anonyme.

Une closure est une fonction qui capture des variables de son environnement et peut les utiliser même après la fin de la fonction d'origine.

Elle permet de garder un état entre plusieurs appels et est souvent utilisée pour les callbacks, goroutines ou fonctions configurables.

Exemple :

```
package main
```

```
import "fmt"
```

```
func compteur() func() int {  
    i := 0  
    return func() int {  
        i++  
    }  
}
```



```

        return i
    }
}

func main() {
    c := compteur()

    fmt.Println(c()) // 1
    fmt.Println(c()) // 2
    fmt.Println(c()) // 3
}

```

Boucle for en GO

Go n'a qu'une seule boucle : for.

Exemple classique :

```

for i := 0; i < 10; i++ {
    fmt.Println(i)
}

```

Boucle type "while" :

```

for x < 10 {
    x++
}

```

Boucle infinie :

```

for condition {
}

```

fallthrough dans switch

fallthrough est un mot-clé utilisé dans les switch pour forcer l'exécution du cas suivant, même si le cas actuel a déjà été exécuté.

Exemples :

Sans fallthrough :

```

switch x {
case 1:
    fmt.Println("un")
case 2:
    fmt.Println("deux")
}

```

```
default:
    fmt.Println("autre")
}
```

Résultat si x = 1 return « un »

Avec fallthrough :

```
switch x {
case 1:
    fmt.Println("un")
    fallthrough
case 2:
    fmt.Println("deux")
default:
    fmt.Println("autre")
}
```

Résultat si x = 1 return « un », « deux », « autre »

Slices et copie

Quand on copy un slices qu'est-ce qu'on copy ?

- ➔ On copie la structure du slice (pointeur + len + cap)
- ➔ Pas les données elles-mêmes

Exemple avec la définition d'une struct Produit avec ces attributs et le faites qu'on puisse instancier la même structure plusieurs fois comme les notions objets avec l'instanciation d'un Classe Produit.

On peut soit l'instancier avec sans ajouter les noms des attributs peut être moins claire. Ou alors on initialise les attributs un par un.

```

// Value receiver : travaille sur une COPIE → ne modifie pas l'original
// Nommage : (p Produit) → convention : 1-2 lettres du type
func (p Produit) Description() string {
    | statut := "indisponible"
    | if p.Actif && p.Stock > 0 {
    |     statut = "disponible"
    | }
    return fmt.Sprintf("%s - %.2f€ (Stock: %d)", p.Nom, p.Prix, statut, p.Stock)
}

func (p Produit) PrixTTC() float64 {
    return p.Prix * 1.20 // TVA 20%
}

// Pointer receiver : travaille sur le VRAI objet → MODIFIE l'original
// Utiliser * quand on veut modifier ou quand la struct est grande
func (p *Produit) AppliquerReduction(pct float64) {
    if pct < 0 || pct > 100 {
        return // validation
    }
    p.Prix = p.Prix * (1 - pct/100)
}

func (p *Produit) Vendre(quantite int) error {
    if quantite > p.Stock {
        return fmt.Errorf("stock insuffisant : %d demandé, %d disponible", quantite, p.Stock)
    }
    p.Stock -= quantite
    return nil
}

// Utilisation - Go gère automatiquement & et *
prod := Produit{Nom: "Clavier", Prix: 89.99, Stock: 20, Actif: true}
fmt.Println(prod.Description()) // value receiver
prod.AppliquerReduction(10) // pointer receiver - Go fait &prod auto
fmt.Printf("Prix réduit: %.2f\n", prod.Prix) // 80.99

err := prod.Vendre(5)
if err != nil { fmt.Println(err) }
fmt.Println(prod.Stock) // 15

```

Points clés :

- Slices : dynamiques, append/copy/range | Maps : clé-valeur, idiome val, ok := m[k]
- Pointeurs : &x adresse, *p déréférence, pointer receiver pour modifier structs

Structs et POO en Go

Exemple de la **notion/utilisation** de structure (composition en GO).

Structure imbriquée

Ce n'est pas de l'héritage ! Mais bien de la composition.

Exemple avec structure Utilisateur avec en attribut les informations d'un utilisateur (id, prenom, email, age, interne, mdp...)

Go n'a pas d'héritage.

On utilise la composition (embedding).

Exemple :

```
type Adresse struct {  
    Ville string  
}
```

```
type Utilisateur struct {  
    Nom    string  
    Adresse  
}
```

On "imbrique" des structures comme des blocs Lego.

Points clés :

- Structs + méthodes + embedding = alternative à l'héritage

Exemple :

```
type Produit struct {  
    Nom string  
    Prix float64  
}
```

Instanciation :

```
p1 := Produit{"Banane", 2.5}
```

```
p2 := Produit{  
    Nom: "Pomme",  
    Prix: 3.0,  
}
```

Struct Tags – métadonnées pour JSON et plus

Permettent d'ajouter des métadonnées.

Struct tags → `json:'nom,omitempty'` — `json:'-'` pour ne pas sérialiser

Go gère auto & et * pour les méthodes → p.MethodePtr() == (&p).MethodePtr()

```
type Utilisateur struct {
    ID      int    `json:"id"           // sérialisé comme "id"
    Prenom  string `json:"prenom"
    Email   string `json:"email,omitempty" // omis si vide ("")
    Age     int    `json:"age,omitempty"  // omis si 0
    Interne string `json:"-'"           // JAMAIS sérialisé
    MDP     string `json:"-'"           // mot de passe : jamais !
}

// Exemple avec JSON (anticipation Jour 3)
import "encoding/json"

u := Utilisateur{ID: 1, Prenom: "Alice", Email: "alice@go.dev", Age: 30}
data, _ := json.Marshal(u)
fmt.Println(string(data))
// {"id":1,"prenom":"Alice","email":"alice@go.dev","age":30}

// Unmarshal : JSON → struct
jsonStr := `{"id":2,"prenom":"Bob"}`
var u2 Utilisateur
json.Unmarshal([]byte(jsonStr), &u2)
fmt.Println(u2.Prenom) // Bob
```

Visibilité en GO

Majuscule → exporté (public)

Minuscule → non exporté (privé package)

Commence par une majuscule => accessible partout (EXPORTER).

Minuscule pas EXPORTER.

La Casse en GO :

En Go, la casse (majuscule/minuscule) est très importante car elle détermine la visibilité des éléments.

Un nom qui commence par une majuscule est public/exporté et accessible depuis d'autres packages.

Ex :

```
type Personne struct {}
func Afficher() {}
```

Un nom qui commence par une **minuscule** est **privé** au package courant.

Ex :

```
type personne struct {}
func afficher() {}
```

Camel Case debute avec une minuscule : privé
Pascal Case debute par une majuscule : partagé

Defer – garantir l'exécution d'une fonction

C'est comme un post-it

Gestion des erreurs

Permet de toute façon l'exécution même si erreur ou pas

defer exécute une fonction à la fin de la fonction courante.

Utile pour cleanup (fermer fichier, connexion...)

Exemple :

```
func test() {  
    defer fmt.Println("fin")  
    fmt.Println("début")  
}
```

Résultat :

début

fin

Package standard incontournables

fmt → affichage (Println, etc.)

strings → manipulation de chaînes

strconv → conversions

math → fonctions mathématiques

sort → tri

time → temps (time.Now(), time.Sleep, time.Second)